

LLM and data-driven services

LLM과 데이터 기반 서비스

03 RAG와 Langchain

목차 데이터 기반 서비스

1. RAG의 개념
2. Langchain 소개
3. 로컬 LLM

1. RAG의 개념



정보 검색과 생성 과정을 결합하여 보다 정확한 응답을 생성하는 방법론

Retrieval Augmented Generation

검색

증강된

생성

- 정보를 검색하고 이를 결합하여 생성 AI를 사용하는 방법



RAG가 없는 경우 발생하는 문제점

미리 학습된 데이터만 사용

정보가 없으면 허위 정보를 생성하여 제공

신뢰할 수 없는 출처로부터 응답을 생성



RAG를 적용하며 기대하는 효과들

최신 정보도 활용하여 응답에 사용

필요한 정보를 제공하여 응답에 사용

정보의 출처를 통제할 수 있음



지금까지 배운 방법들도 RAG 방법론을 적용한 사례

주어진 파일의 데이터를 추출하여 응답을 생성

필요한 데이터를 관계형 데이터베이스에서 추출하고 응답을 생성

인터넷 검색을 통해 최신 정보를 가져오고 응답을 생성



SQL을 이용한 응답 생성 과정

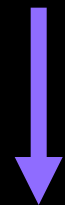
LLM과 데이터 기반 서비스

RAG와 Langchain

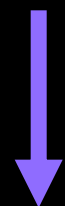
RAG의 개념

```
def query_db(query: str) -> list:
```

쿼리를 DB에 전달하여 데이터를 반환하는 함수



응답에 필요한 데이터를 검색한 결과



검색 결과를 포함하여 다시 응답을 생성

Retrieval
검색

- DB에서 쿼리를 실행하여 관련 정보를 검색
- 사용자 요청에 필요한 데이터를 가져옴

Augmented
증강

- 검색된 데이터를 결합하여 새로운 입력으로 사용

Generation
생성

- 사용자의 입력과 검색된 결과를 기반으로 다시 응답을 생성



1장에서 배운 장단점과 동일하며 각 기능도 이미 구현해본 내용

최신 정보도 활용하여 응답에 사용



웹 크롤러를 이용한 검색

필요한 정보를 제공하여 응답에 사용



SQL을 이용한 검색

정보의 출처를 통제할 수 있음



사용할 함수에 따라 제어 가능



검색기

(Retrieval Component)

- 사용자의 질문 관련 데이터를 데이터베이스나 웹에서 실시간으로 검색하는 역할.
- 검색 성능에 따라 RAG의 성능이 결정됨
- 앞에서 개발해본 함수들이 검색기의 일종

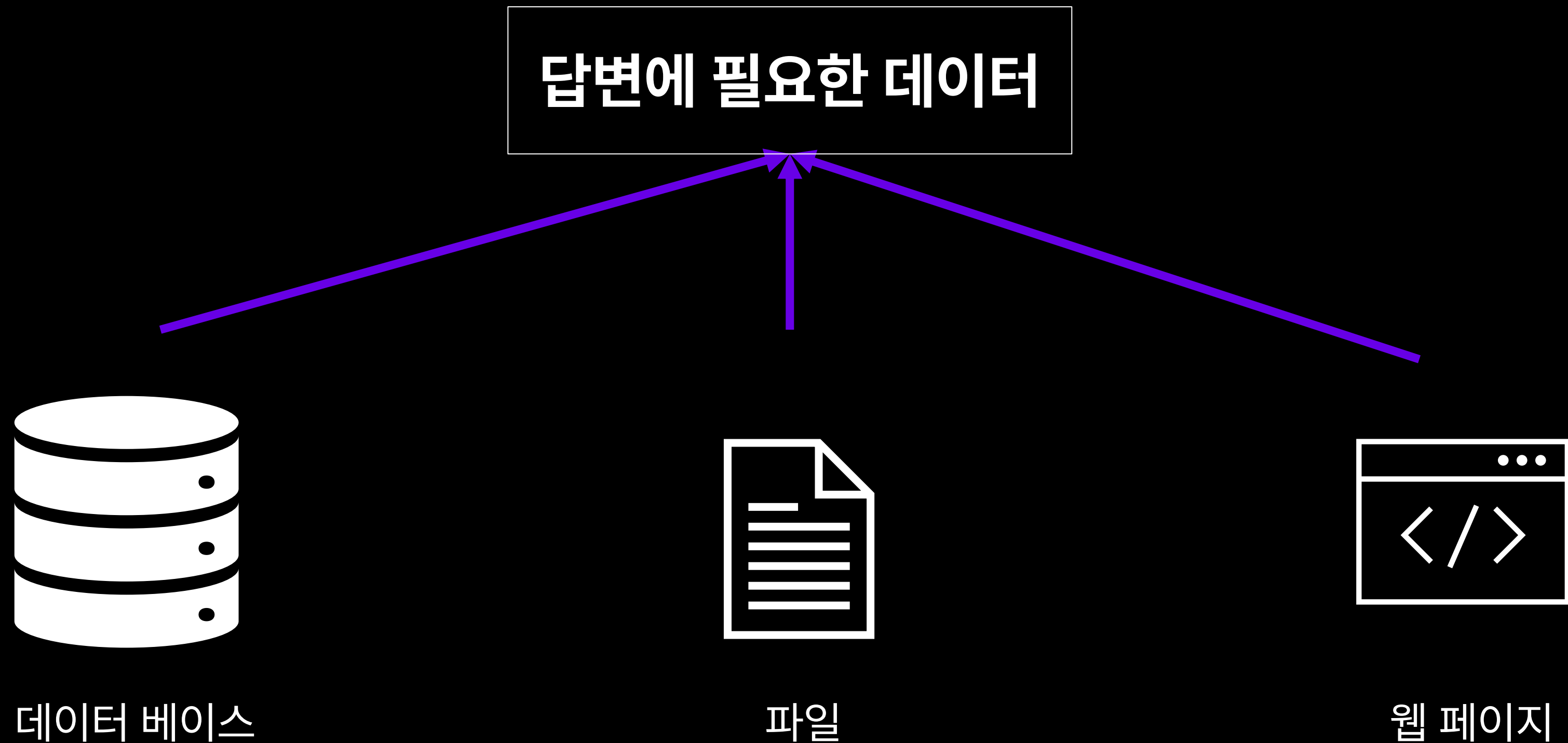
생성기

(Generative Component)

- 검색된 데이터를 바탕으로 사용자의 질문에 맞는 답변을 생성하는 LLM
- 기존 LLM이 여기에 해당하며, 검색기에서 찾은 정보를 더해 개선된 답변을 생성



LLM이 학습하지 않은 외부 데이터에서 필요한 데이터를 검색하여 가져옴



검색기가 찾아온 데이터를 이용하여 더 적절한 답변을 생성함

답변에 필요한 데이터



생성 모델

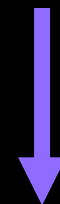


ChatGPT

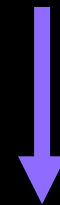


RAG가 최근 주목받는 배경

ChatGPT를 시작으로 생성 AI로 많은 작업을 자동화 (엑셀 함수 등)



정보를 얻는 용도로 사용하기에는 할루시네이션 등의 문제가 존재

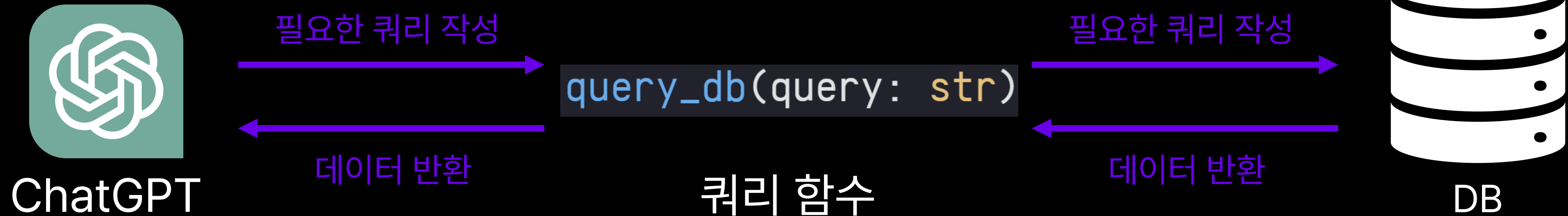


데이터 기반으로 정확한 언어모델의 필요성 대두

2. Langchain 소개

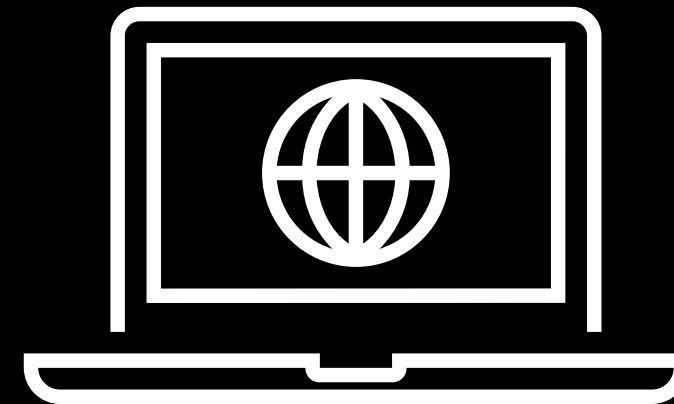
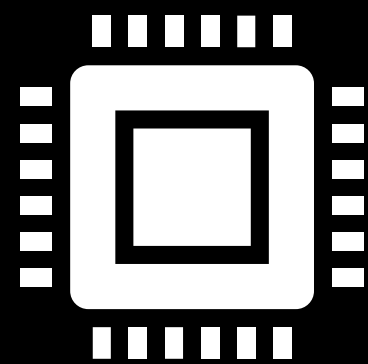


지금까지는 정보 검색과 텍스트 생성을 하나의 흐름으로 처리



- ChatGPT API와 DB와 사용자의 응답처리가 서로 의존적으로 진행되고 있음
- 데이터베이스 쿼리나 호출을 변경할 때도 코드 전체를 수정해야 할 수 있음
- 확장성과 유지보수를 위해 각각을 독립적으로 관리할 필요가 있음

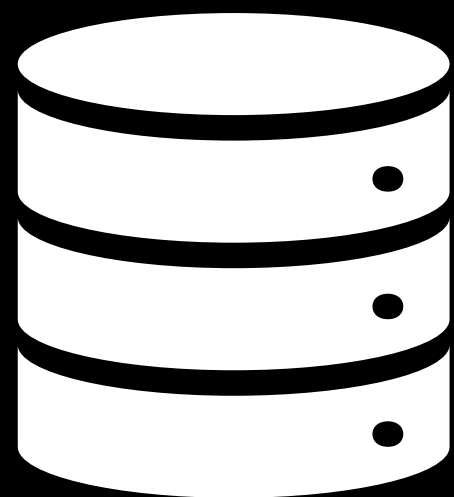
각 부품이 독립되어 있어서 교체나 관리가 용이한 현대 컴퓨터의 예시



- 각 부품들이 컴퓨터라는 시스템을 구성하지만, 각 부품은 서로 독립적
- 프로그램도 이와 같이 기능들을 나눠 서로 독립적으로 만드는 과정을 모듈화라고 함



모든 필요한 기능을 직접 구현하는 것은 비효율적



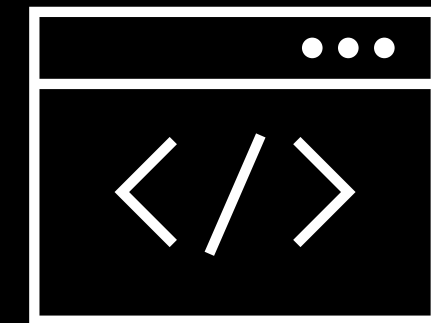
SQL문법
관계형DB
제한사항

데이터 베이스



파일처리방법
관련 업데이트

파일

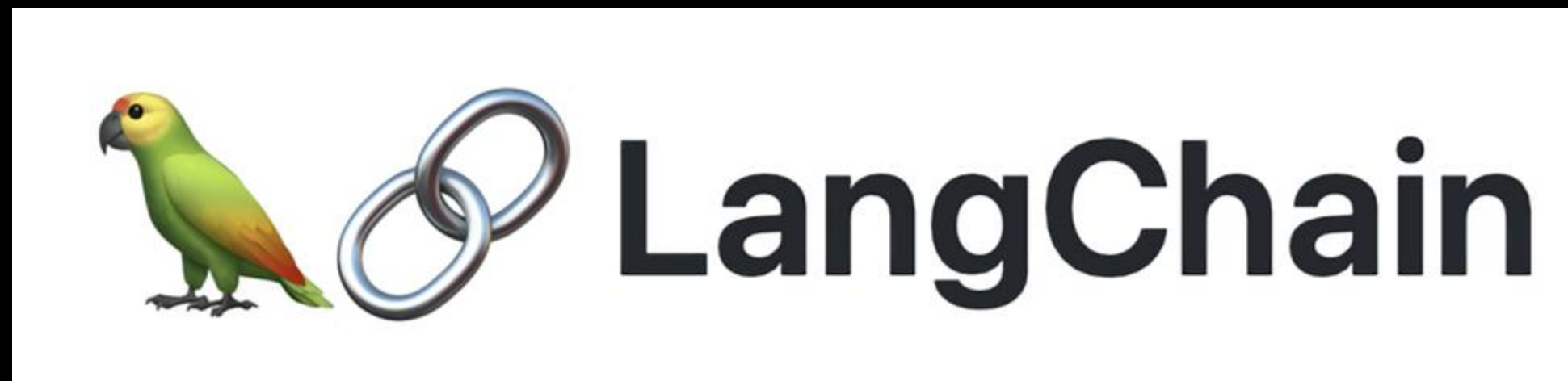


웹 서비스 구조
HTML
크롤러

웹 페이지

- 각각에 필요한 함수를 직접 구현하고 이에 맞게 전처리 후처리도 추가해야 함
- 각 기능을 구현하려면 각 도메인의 지식을 요구함
- 새로운 기능을 넣는 것이 오래 걸리고 유지관리가 어려움

구현을 위해 다양한 구성 요소와 기능을 제공하는 라이브러리



- 각 기능을 쉽게 구현하고 서로 간단하게 연동
- 대부분의 작업이 툴로 이미 제작되어 있음



```
# 시스템 프롬프트 정의 (템플릿 형식 유지)
```

```
SYSTEM_PROMPT = """너는 사람들의 다양한 질문에 대답해주는 챗봇이야.
```

```
대화시작시간: {conversation_start_time}
"""
```

```
chat_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", SYSTEM_PROMPT),
        ("user", "{user_input}"),
    ]
)
```

```
messages = chat_prompt.format_messages(
    user_input="지금 몇 시야?",
    conversation_start_time=datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
)
pprint(messages)
```

대화 시작 시간



```
# 시스템 프롬프트 정의 (템플릿 형식 유지)
```

```
SYSTEM_PROMPT = """너는 사람들의 다양한 질문에 대답해주는 챗봇이야.  
대화시작시간: {conversation_start_time}  
"""
```

```
chat_prompt = ChatPromptTemplate.from_messages(  
    [  
        ("system", SYSTEM_PROMPT),  
        ("user", "{user_input}"),  
    ]  
)
```

```
messages = chat_prompt.format_messages(  
    user_input="지금 몇 시야?",  
    conversation_start_time=datetime.now().strftime("%Y-%m-%d %H:%M:%S"),  
)  
pprint(messages)
```

사용자 메시지



```
# 시스템 프롬프트 정의 (템플릿 형식 유지)
SYSTEM_PROMPT = """너는 사람들의 다양한 질문에 대답해주는 챗봇이야.
대화시작시간: {conversation_start_time}
"""

chat_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", SYSTEM_PROMPT),
        ("user", "{user_input}"),
    ]
)

messages = chat_prompt.format_messages(
    user_input="지금 몇 시야?",
    conversation_start_time=datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
)
pprint(messages)
```

```
[ SystemMessage(content='너는 사람들의 다양한 질문에 대답해주는 챗봇이야.
대화시작시간: 2024-10-24 01:49:31' ),
  HumanMessage(content='지금 몇 시야?')]
```



프롬프트: 입력 형식

```
# 프롬프트
chat_prompt = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_PROMPT),
    ("user", "{user_input}"),
])
```

LLM: 언어모델

```
# OpenAI LLM 정의
llm = ChatOpenAI(
    api_key="sk-",
    model="gpt-4o-mini",
)
```

출력을 특정 형식으로 파싱

```
# 문자열 파서
StrOutputParser()
```



체인으로 각 요소를 하나로 연결

파이프라인 연결

```
chain = chat_prompt | llm | StrOutputParser()
```

```
def chat(user_input: str) -> str:
    msg = {
        "user_input": user_input,
        "conversation_start_time": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    }
    print(msg)
    return chain.invoke(msg)

resp = chat("지금 시간 알려줘")
print(resp)
```



각 기능들을 직접 구현하고 이를 연동하는 과정도 직접 구현

검색으로 URL추출

```
def google_search(query: str, num_results=10) -> list:
    "Google 검색 결과에서 상위 URL을 가져오는 함수"
    try:
        urls = [url for url in search(query, num_results=num_results)]
        return urls[:num_results]
    except Exception as e:
        print(f"Error during Google search: {e}")
        return []
```

```
def extract_text(url: str) -> str:
    try:
        headers = {"User-Agent": "Mozilla/5.0"}
        response = requests.get(url, headers=headers, timeout=10)
        response.raise_for_status()
        soup = BeautifulSoup(response.content, "html.parser")
        text = soup.get_text(separator="\n", strip=True)
        return text

    except Exception as e:
        print(f"Error scraping {url}: {e}")
        return ""
```

URL에서 정보 추출

검색 결과를 텍스트로 변환

```
def search_web(query: str) -> List[str]:
    """인터넷에서 query를 검색하고 상위 4개 페이지의 텍스트를 모아서 반환하는 함수

    Args:
        query (str): 웹에 검색할 문자열. 관련 정보를 웹에서 찾기 위해 검색할 적절한
            검색어로 설정해야 함.

    Returns:
        list: 상위 4개 페이지의 텍스트 리스트
    """
    urls = google_search(query)
    text_list = []

    count = 0

    for url in urls:
        page_text = extract_text(url)
        if page_text:
            page_text = post_process(page_text)
            page_text = f"{count + 1}. {url}:\n{page_text}\n"
            text_list.append(page_text)
            count += 1

    if count >= 4:
        break
    return text_list
```



에이전트라는 것으로 툴을 조립하는 수준

전체 코드

```
from langchain.agents import AgentType, Tool, initialize_agent
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_openai import ChatOpenAI

# LLM 초기화
llm = ChatOpenAI(api_key="sk-")

# DuckDuckGo 검색 툴 초기화
search = DuckDuckGoSearchRun()

# 에이전트 초기화 - ZERO_SHOT_REACT_DESCRIPTION 사용
agent = initialize_agent(
    tools=[search],
    llm=llm,
    agent_type=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    agent_kwargs={"prefix": "너는 사용자에게 질문에 답변하는 챗봇이야."},
)

# 에이전트를 통해 질문 실행 후, 답변 받기
result = agent.invoke("어제 서울 날씨 어땠어?")

# 답변 출력
print(result.get("output"))
```

LLM 초기화

검색엔진

하나로 조립

사용자 입력

답변 출력



답변을 만들기까지 과정도 쉽게 볼 수 있음

```
from langchain.agents import AgentType, Tool, initialize_agent
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_openai import ChatOpenAI

# LLM 초기화
llm = ChatOpenAI(api_key="sk-")

# DuckDuckGo 검색 툴 초기화
search = DuckDuckGoSearchRun()

# 에이전트 초기화 - ZERO_SHOT_REACT_DESCRIPTION 사용
agent = initialize_agent(
    tools=[search],
    llm=llm,
    agent_type=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    agent_kwargs={"prefix": "너는 사용자에게 질문에 답변하는 챗봇이야."},
)

# 에이전트를 통해 질문 실행 후, 답변 받기
result = agent.invoke("어제 서울 날씨 어땠어?")

# 답변 출력
print(result.get("output"))
```

(시간 정보와 함께 사용시 정확한 시점을 기준으로 할 수 있음)

> Entering new AgentExecutor chain...

어제 서울 날씨에 대한 정보를 찾기 위해 검색해야겠다.

Action: duckduckgo_search

Action Input: "서울 날씨 2023년 10월 25일"

Observation: 오늘의 날씨 - 오전 (2023년 10월 25일) 입력:2023-10-25 05:32. 10월 25일 수요일, 아침 날씨입니다. 아침 최저기온은 서울 13.0도, 인천 12.0도, 수원 11.0도, 춘천 11.0도, 강릉 13.0도, 청주 11.0도, 대전 11.0도, 전주 12.0도, 광주 11.0도, 대구 11.0도, 부산 12.0도, 제주 14.0도로 예상 ... 올해 10월의 날씨 전망에 대해 알아보겠습니다. 1. 작년 2023년 10월의 날씨. 최저기온: 5.4도 (23/10/22) 최고기온: 24.6도 (23/10/2) 작년 10월은 지긋지긋한 비에서 벗어나 강수량이 적었으며, 맑은 날이 많아서 가을하늘을 느낄 수 있는, 가을 다운 날씨였던 것 같습니다 ... 이 보고서는 서울특별시의 과거 날씨를 보여주며 2023년 11월의 날씨 기록을 제공합니다. 2023년 11월 서울특별시 온도 기록을 포함하여 사용 가능한 모든 과거 날씨 데이터 시리즈를 제공합니다. 그 래프를 클릭하여 연도별 및 일별 보고서를 드릴다운할 수 있습니다. 서울 강남 송파구 오늘의 날씨 2023년 10월 25일 (21시 30분 현재) 어제. 최저기온 12도 (오전), 최고기온 19도 (낮) 오늘. 최저기온 13도 (오전), 최고기온 24도 (낮) 어제보다 1도 높은 최저기 온, 어제보다 5도 높은 최고기온입니다 아침에 최저기온 영상 13도이고 낮에 ... 기상청에 따르면 10월 중순인 오는 16일 (서울 기준)의 최저기온은 17도, 최고 26도를 기록한다. 기상청. 이는 지난해 같은 날 (2023년 10월 16일, 서울 기준) 최저기온 11도, 최고 20도를 기록한 것과 비교해보면 확실한 '이상기후'로 보인다. 실제로 지난달 30일 국립기상 ...

Thought:어제 서울의 날씨에 대한 정보를 찾았다. 어제, 즉 10월 24일의 최저 기온은 12도였고, 최고 기온은 19도였다. 날씨는 비교적 온화했음을 알 수 있다.

Final Answer: 어제 서울의 날씨는 최저 12도, 최고 19도로 온화했습니다.

> Finished chain.

어제 서울의 기온은 10.2도였습니다.

3. 로컬 LLM



좋은 성능을 보이지만 매우 거대한 모델

GPT 3
(2020년)

약 1750억 개의 **파라미터**

16비트(2바이트) 실수

$$175,000,000,000 \times 2 = 350,000,000,000 \\ = 350 \text{ GB}$$



24 GB

× 14.5

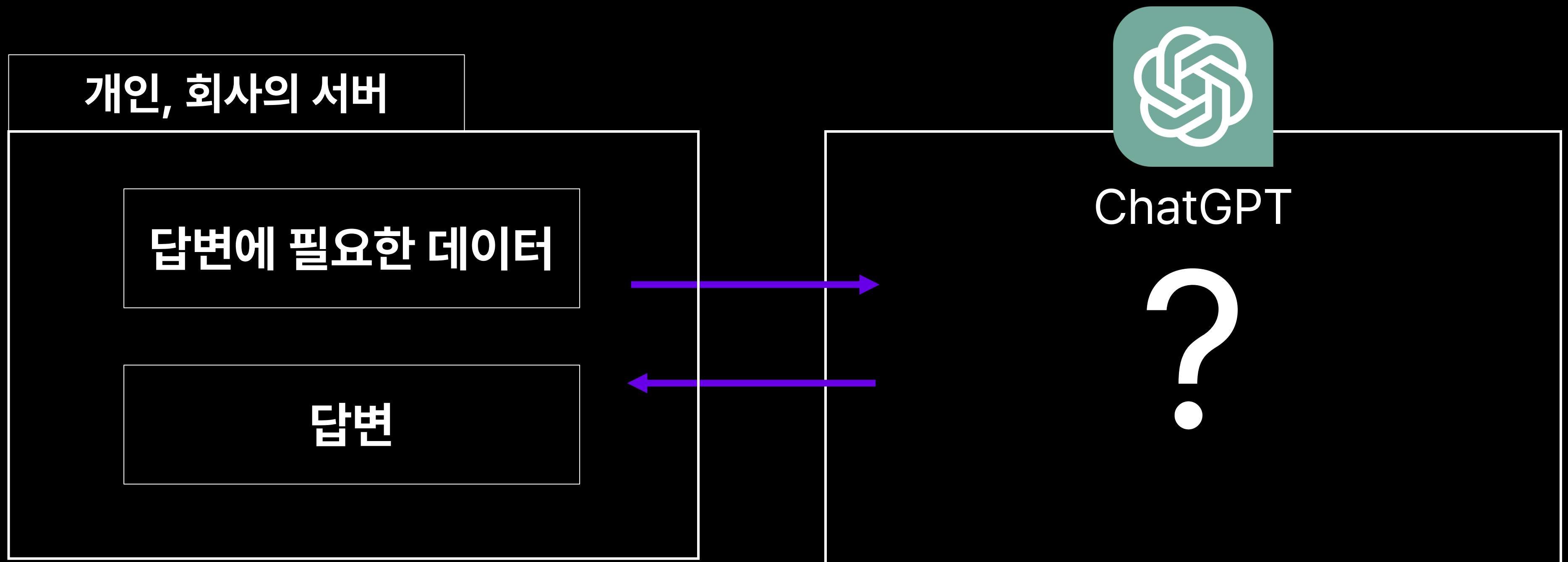
이전 버전에 비해 더 거대한 모델로 호출에 따라 비용이 발생

Model	Pricing
gpt-4o	\$2.50 / 1M input tokens
	\$1.25 / 1M cached** input tokens
	\$10.00 / 1M output tokens

API 비용표



중요한 데이터도 API를 통해 전송하면서 주의가 필요





중요한 데이터도 API를 통해 전송하면서 주의가 필요

생성 모델



ChatGPT



- 최근에는 구체적인 동작 방법이나 구조에 대해 공개하지 않음
- API를 통해 사용할 수 있지만 내부의 동작은 알 수 없음
- 중요한 기밀 데이터를 전송하는데 주의가 필요



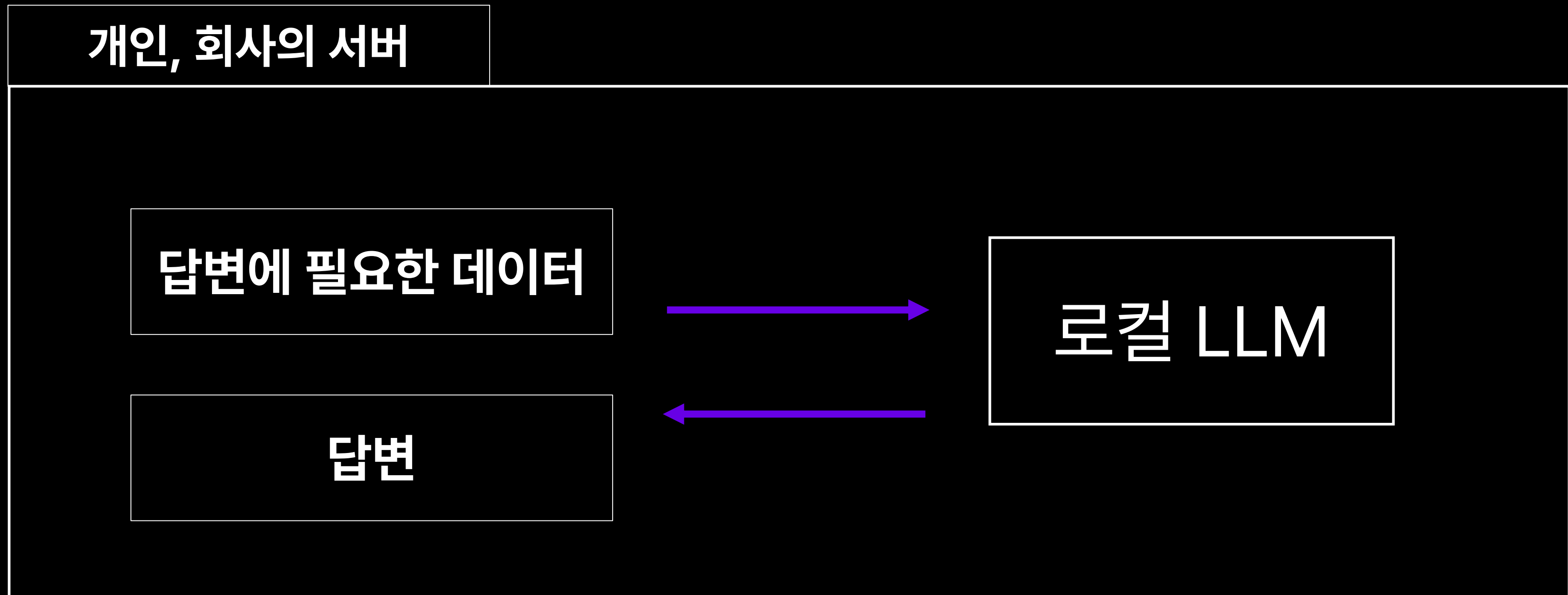
외부의 서버가 아닌 내부에서 직접 모델을 실행하여 사용

Llama



- 공개된 코드
- 직접 학습시킬 수 있음
- 폐쇄된 환경에서도 사용 가능

외부로 데이터가 전송되지 않고 언어모델을 사용 가능





구현 난이도가 높고 많은 하드웨어 자원이 필요

Llama 3.2



- 약 90B(900억) 개의 파라미터
= 180GB

- 동작에 필요한 컴퓨팅 자원을 확보하는 비용이 발생
- 직접 구현하기 위한 AI 엔지니어가 필요
- 서버의 구조부터 구현이 필요



성능은 유지하면서 모델에 필요한 컴퓨팅 자원을 줄이기 위한 노력

Llama 3.2

경량화 버전

- 3B: 6GB

- 1B: 2GB



아직까지는 직접 구현을 통해 ChatGPT보다 높은 성능으로 사용하기 어려움

```
resp = chain.invoke("키오스크 설문조사 결과를 알려줘")  
print(resp)
```

설문결과는 다음과 같이 나왔다.

1. 20대: 전체 2%가 5성, 22%가 4성이 반응한 것으로 드러났습니다.
2. 30대: 전체 6%가 5성, 27%가 4성이 반응한 것으로 드러났습니다.
3. 40대: 전체 17%가 5성, 43%가 4성이 반응한 것으로 드러났습니다.
4. 50대: 전체 30%가 5성, 51%가 4성이 반응한 것으로 드러났습니다.
5. 60대 이상: 전체 32%가 5성, 57%가 4성이 반응한 것으로 드러났습니다.

이번 조사 결과로 보았을 때 가장 만족도가 높은 세대는 50대라고 결론이 났다.

이 소식은 이용자들에게 전달되어야 한다.

- 모델의 답변 제어, 학습, 사용 데이터 셋에 따라 크게 성능이 달라짐
- 간단한 구현을 통해서도 ChatGPT에 비슷한 성능을 구현할 수 없음